



FACULDADE DE TECNOLOGIA SENAI GASPAR RICARDO JÚNIOR

**DESENVOLVIMENTO DE UM SISTEMA SUPERVISÓRIO WEB PARA
MONITORAMENTO INDUSTRIAL UTILIZANDO OPC-UA,
NODE-RED E NEXT.JS**

**DEVELOPMENT OF A WEB-BASED SUPERVISORY SYSTEM FOR
INDUSTRIAL MONITORING USING OPC-UA, NODE-RED AND
NEXT.JS**

Luciano de Jesus Menezes Junior ¹

RESUMO

Este trabalho descreve a concepção e implementação de um sistema supervisório Web voltado ao monitoramento e controle de processos industriais, unindo tecnologias modernas de desenvolvimento Web a protocolos de comunicação industrial consolidados. A solução adota uma arquitetura distribuída em quatro camadas: um servidor OPC-UA desenvolvido em Python para a geração de variáveis de processo (temperatura, pressão e status de operação), o middleware Node-RED responsável pela aquisição e encaminhamento de dados via protocolo OPC-UA, um backend em Express.js (Node.js) para armazenamento de históricos e tratamento de alarmes, e uma interface Web em Next.js com dashboards em tempo real, gráficos de tendência e tela de diagnóstico de alarmes. Um segundo backend em .NET, integrado a um banco de dados PostgreSQL, gerencia o cadastro de usuários, dispositivos e sensores por meio de uma API RESTful. A metodologia compreendeu a construção de um pipeline completo de aquisição de dados industriais (OPC-UA → Node-RED → Express → Next.js), observando boas práticas de alarmística industrial e os princípios de interfaces de alta performance preconizados pela norma ISA-101. Os resultados evidenciam a viabilidade da solução para aplicações supervisórias em ambientes educacionais e

de prototipagem, com suporte a monitoramento em tempo real, registro de histórico de processo e gestão de alarmes.

Palavras-chave: Sistema Supervisório; OPC-UA; Node-RED; IoT Industrial; Next.js; Alarmes Industriais.

ABSTRACT

This paper describes the design and implementation of a Web-based supervisory system for monitoring and controlling industrial processes, combining modern Web development technologies with well-established industrial communication protocols. The solution adopts a distributed four-layer architecture: an OPC-UA server developed in Python for generating process variables (temperature, pressure, and operating status), the Node-RED middleware responsible for data acquisition and routing via the OPC-UA protocol, an Express.js (Node.js) backend for historical data storage and alarm handling, and a Next.js Web interface featuring real-time dashboards, trend charts, and an alarm diagnosis screen. A second .NET backend, integrated with a PostgreSQL database, manages the registration of users, devices, and sensors through a RESTful API. The methodology encompassed the construction of a complete industrial data acquisition pipeline (OPC-UA → Node-RED → Express → Next.js), following industrial alarm management best practices and the high-performance interface principles set forth by the ISA-101 standard. The results demonstrate the feasibility of the solution for supervisory applications in educational and prototyping environments, supporting real-time monitoring, process history recording, and alarm management.

Keywords: Supervisory System; OPC-UA; Node-RED; Industrial IoT; Next.js; Industrial Alarms.

1 Introdução

A transformação digital no setor produtivo tem acelerado a adoção de tecnologias associadas à Internet das Coisas Industrial (IIoT) e a sistemas supervisórios baseados em Web, promovendo a substituição progressiva das soluções SCADA tradicionais por plataformas mais flexíveis, acessíveis e interoperáveis (AGUIAR, 2023). Nesse cenário, a combinação entre protocolos de comunicação industrial, como o OPC-UA,

e frameworks contemporâneos de desenvolvimento Web representa uma abordagem promissora para a construção de interfaces homem-máquina (IHM) de alto desempenho.

O protocolo OPC-UA (Open Platform Communications – Unified Architecture) consolida-se como referência para a troca de informações entre equipamentos industriais, fornecendo interoperabilidade, segurança e modelagem de dados orientada a objetos (OPC FOUNDATION, 2017). Quando associado a ferramentas de middleware como o Node-RED, torna-se viável criar pipelines de aquisição de dados que conectam o chão de fábrica a aplicações Web em tempo real.

O objetivo deste trabalho é apresentar o desenvolvimento de um sistema supervisório Web completo, capaz de:

- Adquirir dados de um servidor OPC-UA simulado por meio do middleware Node-RED;
- Armazenar e processar variáveis de processo em um backend Express.js;
- Implementar mecanismos de alarmística industrial com reconhecimento e registro histórico;
- Exibir informações em uma interface Web com dashboards em tempo real, gráficos de tendência e telas de diagnóstico, em conformidade com os princípios da norma ISA-101;
- Gerenciar cadastros de usuários, dispositivos e sensores por meio de uma API CRUD em .NET integrada a um banco PostgreSQL.

O projeto foi realizado no âmbito da disciplina de Interfaces Industriais, contemplando os módulos de comunicação industrial, backend supervisório, alarmística e design de IHM.

2 Revisão de Literatura

2.1 Sistemas Supervisórios e SCADA

Os sistemas supervisórios constituem ferramentas fundamentais para o monitoramento e o controle de processos industriais. Historicamente implementadas sob o

modelo SCADA (Supervisory Control and Data Acquisition), essas soluções vêm migrando para arquiteturas Web, o que amplia o acesso remoto e confere maior maleabilidade à construção de interfaces (AGUIAR, 2023).

Conforme apontam Galloway e Hancke (2013), os sistemas SCADA contemporâneos tendem a incorporar padrões abertos de comunicação e interfaces baseadas em navegador, diminuindo a dependência de softwares proprietários e favorecendo a integração com ecossistemas de IoT.

2.2 OPC-UA como Padrão de Comunicação Industrial

O OPC-UA é um protocolo de comunicação independente de plataforma, concebido especificamente para ambientes industriais. Ao contrário de versões anteriores, como o OPC-DA (restrito ao sistema Windows por depender de COM/DCOM), o OPC-UA oferece comunicação segura, modelagem de dados orientada a objetos e suporte a múltiplos padrões de transporte (OPC FOUNDATION, 2017).

“A arquitetura OPC-UA foi projetada para fornecer uma infraestrutura robusta e segura para a troca de dados em aplicações industriais, eliminando as limitações de plataforma e promovendo a interoperabilidade entre dispositivos de diferentes fabricantes” (OPC FOUNDATION, 2017).

A estrutura do OPC-UA fundamenta-se em nós (*nodes*), que representam variáveis, objetos e métodos. Cada nó possui um identificador único (NodeId) que viabiliza leitura e escrita de valores de modo padronizado.

2.3 Node-RED como Middleware Industrial

O Node-RED é uma ferramenta de programação visual baseada em fluxos, originalmente desenvolvida pela IBM para aplicações de IoT. Sua arquitetura possibilita a criação de pipelines de aquisição de dados que interligam nós de entrada (sensores e protocolos), de processamento (transformação e filtragem) e de saída (APIs e bancos de dados) (NODE-RED, 2024).

No contexto industrial, o Node-RED atua como middleware entre dispositivos de campo e sistemas supervisórios, abstraindo a complexidade dos protocolos de comunicação e viabilizando a integração ágil de novos dispositivos ao pipeline de dados (RODRIGUES, 2024).

2.4 APIs RESTful para Sistemas IoT

De acordo com Rodrigues (2024), a ampliação de funcionalidades em sistemas IoT está diretamente relacionada à integração com APIs HTTP, que oferecem maior flexibilidade e a possibilidade de controle remoto dos dispositivos. A adoção de arquiteturas REST assegura a interoperabilidade entre diferentes plataformas e tecnologias.

Aguiar (2023) complementa que a Internet das Coisas pode ser definida como uma rede de objetos físicos conectados à internet, habilitados a interagir por meio de sensores e softwares, sendo as APIs RESTful peças fundamentais para a conexão entre o mundo físico e o digital.

2.5 Alarmística Industrial e Norma ISA-101

A norma ISA-101 (*Human Machine Interfaces for Process Automation Systems*) estabelece diretrizes para o projeto de interfaces industriais de alta performance. Entre seus princípios, destacam-se o uso criterioso de cores, a hierarquia de telas e o destaque adequado para alarmes e situações anormais (INTERNATIONAL SOCIETY OF AUTOMATION (ISA), 2015).

A gestão de alarmes em sistemas supervisórios envolve a classificação por prioridade, o reconhecimento pelo operador e o registro histórico para análise de tendências e diagnóstico de falhas.

3 Metodologia

O sistema foi concebido e desenvolvido seguindo uma arquitetura em quatro camadas, conforme ilustrado na Figura 1. O fluxo de dados percorre o seguinte caminho: OPC-UA Server → Node-RED → Backend Express → Frontend Next.js.

Figura 1: Arquitetura geral do sistema supervisorio Web.

Diagrama de Arquitetura do Sistema

```

OPC-UA Server (Python : 4840)
    ↓ protocolo OPC-UA
Node-RED (Middleware : 1880)
    ↓ HTTP POST
Backend Express (Node.js : 8080)
    ↓ API REST
Frontend Next.js (React : 3000)

Backend .NET (5071) + PostgreSQL (5432)
(CRUD de usuários, dispositivos e sensores)
  
```

3.1 Camada 1 – Servidor OPC-UA (Python)

O servidor OPC-UA foi construído em Python com o auxílio da biblioteca `python-opcua`. Ele simula um processo industrial por meio de três variáveis:

- **Temperatura** (`ns=2; i=2`, Float): variação aleatória em torno de 25 °C, com limites entre 5 °C e 95 °C;
- **Pressão** (`ns=2; i=3`, Float): variação aleatória em torno de 2,5 bar, com limites entre 0,2 e 6,0 bar;
- **Status de Operação** (`ns=2; i=4`, Boolean): estado ligado/desligado com probabilidade de alternância de 5%.

Os valores são publicados a cada 1 segundo no endpoint `opc.tcp://localhost:4840`, permitindo leitura e escrita por clientes OPC-UA.

3.2 Camada 2 – Middleware Node-RED

O Node-RED atua como middleware de aquisição de dados, executando fluxos que:

1. Conectam-se ao servidor OPC-UA na condição de cliente;
2. Realizam a leitura das variáveis de processo (Temperatura, Pressão e Status) a cada 2 segundos;
3. Formatam os dados em objetos JSON com os campos `tag`, `valor`, `unidade` e `nodeId`;
4. Encaminham os dados ao backend Express via requisições HTTP POST para o endpoint `/iot`.

Os fluxos utilizam os nós `node-red-contrib-opcua` para comunicação OPC-UA e nós nativos `http request` para o envio dos dados.

3.3 Camada 3 – Backend Express.js

O backend foi implementado em Node.js com o framework Express.js, operando na porta 8080. Suas responsabilidades incluem:

- **Recepção de dados** (POST `/iot`): recebe leituras provenientes do Node-RED e as armazena em memória;
- **Armazenamento de histórico**: mantém até 500 leituras por tag para consultas posteriores;
- **Gerenciamento de alarmes**: verifica limites configurados e gera alarmes automaticamente;
- **API de consulta**: endpoints GET para valores atuais (`/current`), histórico (`/history/:tag`), alarmes (`/alarms`) e status (`/status`);
- **Escrita OPC-UA**: endpoint POST `/write` para envio de comandos diretamente ao servidor OPC-UA por meio da biblioteca `node-opcua-client`.

Os limites de alarme definidos no backend estão apresentados na Tabela 1.

Tabela 1: Limites de alarme configurados no backend.

Tag	Limite Inferior	Limite Superior	Unidade
Temperatura	10	80	°C
Pressão	0,5	5,0	bar

3.4 Camada 4 – Frontend Next.js

A interface Web foi construída com o framework Next.js (React) e TypeScript, executando na porta 3000. As telas implementadas são:

- **Dashboard Principal** (/): exibe os valores atuais de temperatura, pressão e status de operação em tempo real, com gráficos de tendência (últimos 60 pontos) e banner de alarmes ativos. Atualização automática a cada 3 segundos;
- **Histórico de Processo** (/historico): permite a seleção de tag (Temperatura ou Pressão), período (1, 5 ou 10 minutos) e tipo de gráfico (área ou linha), exibindo estatísticas descritivas e tabela de dados brutos;
- **Alarmes e Diagnóstico** (/alarmes): apresenta resumo de alarmes com filtros por prioridade, status e tag, além de funcionalidades de reconhecimento individual e em lote;
- **Dispositivos** (/devices): CRUD de dispositivos e sensores integrado ao backend .NET;
- **Usuários** (/users): gerenciamento de usuários com autenticação JWT;
- **Login** (/login): autenticação via JWT com chave simétrica.

A biblioteca Recharts foi utilizada para a renderização dos gráficos de tendência (AreaChart e LineChart), e o gerenciamento do estado de autenticação foi feito com a Context API do React.

3.5 Camada Complementar – Backend .NET e PostgreSQL

Um backend em ASP.NET Core (.NET 9) opera na porta 5071 e disponibiliza uma API RESTful para operações CRUD de usuários, dispositivos e sensores/atuadores. O banco de dados PostgreSQL 15 é executado em container Docker, armazenando todos os dados cadastrais do sistema.

3.6 Tecnologias Utilizadas

A Tabela 2 sintetiza as tecnologias empregadas em cada camada do sistema.

Tabela 2: Tecnologias utilizadas no projeto.

Camada	Tecnologia	Porta
Servidor OPC-UA	Python + python-opcua	4840
Middleware	Node-RED v4.1.8	1880
Backend IoT	Node.js + Express.js	8080
Frontend	Next.js + React + TypeScript	3000
Backend CRUD	ASP.NET Core (.NET 9)	5071
Banco de Dados	PostgreSQL 15 (Docker)	5432
Gráficos	Recharts	–
Autenticação	JWT (HS256)	–

4 Resultados e Discussões

O sistema supervisório Web foi implementado com sucesso, atendendo a todos os requisitos estabelecidos pela disciplina de Interfaces Industriais.

4.1 Pipeline de Aquisição de Dados

O pipeline completo OPC-UA → Node-RED → Express → Next.js foi validado com dados simulados. O servidor OPC-UA gera valores aleatórios de temperatura e pressão, reproduzindo o comportamento de um processo industrial. Os dados são coletados pelo Node-RED a cada 2 segundos e remetidos ao backend Express via HTTP POST.

A Tabela 3 ilustra um exemplo de leitura das variáveis de processo em um instante de operação.

Tabela 3: Exemplo de valores de processo monitorados em tempo real.

Variável	Valor	Unidade	NodeId OPC-UA
Temperatura	32,47	°C	ns=2;i=2
Pressão	2,83	bar	ns=2;i=3
Status	Ligado	–	ns=2;i=4

4.2 Dashboard de Monitoramento em Tempo Real

O dashboard principal apresenta os valores das variáveis de processo em cards informativos, acompanhados de gráficos de tendência que evidenciam o comportamento recente do processo. A atualização automática a cada 3 segundos assegura ao operador uma visão contínua do estado da planta.

Os gráficos são renderizados com a biblioteca `Recharts`, empregando o componente `AreaChart` com preenchimento em gradiente para temperatura e pressão, facilitando a identificação visual de desvios e tendências.

4.3 Sistema de Alarmística

O módulo de alarmes realiza a verificação automática de limites para temperatura (10 °C a 80 °C) e pressão (0,5 a 5,0 bar). Quando uma variável ultrapassa os limites configurados, um alarme é gerado automaticamente contendo as seguintes informações:

- Tag da variável, tipo (HIGH/LOW) e prioridade;
- Valor medido e limite violado;
- Timestamp da ocorrência;
- Estado de reconhecimento e resolução.

A tela de alarmes permite ao operador visualizar ocorrências ativas, reconhecê-las de forma individual ou em lote, e consultar o histórico de alarmes encerrados. A resolução automática dos alarmes ocorre quando a variável retorna à faixa normal de operação, em consonância com o princípio de *return-to-normal* usual em sistemas supervisórios industriais.

4.4 Histórico de Processo

A tela de histórico possibilita a consulta de dados armazenados, com seleção de variável, período e tipo de gráfico. O sistema calcula e exibe estatísticas descritivas (valor mínimo, médio e máximo) para o intervalo selecionado, além de disponibilizar uma tabela com as últimas 20 leituras brutas.

O backend armazena até 500 leituras por tag em memória, viabilizando análises de tendência para intervalos de até aproximadamente 17 minutos (considerando a taxa de aquisição de 2 segundos do Node-RED).

4.5 Escrita OPC-UA via Backend

O backend implementa capacidade de escrita direta no servidor OPC-UA por meio do endpoint `POST /write`, utilizando a biblioteca `node-opcua-client`. Essa funcionalidade permite o envio de comandos de controle ao processo simulado, como alteração de setpoints e acionamento de atuadores, tornando o sistema bidirecional.

4.6 Discussão

O sistema desenvolvido demonstra que é viável construir um supervisório industrial funcional utilizando exclusivamente tecnologias Web de código aberto. A arquitetura em camadas promove a separação de responsabilidades e permite a substituição individual de componentes sem impacto nas demais partes do sistema.

A principal limitação identificada é o armazenamento do histórico em memória volátil, o que implica perda dos dados ao reiniciar o backend. Em um cenário de produção, recomenda-se a integração com um banco de dados de séries temporais, como InfluxDB ou TimescaleDB.

A taxa de aquisição de 2 segundos, suficiente para a simulação proposta, pode ser inadequada para processos que exijam resolução temporal na ordem de milissegundos. A substituição do polling HTTP por WebSockets representaria uma melhoria significativa na latência de atualização da interface.

5 Conclusão

Este trabalho apresentou o desenvolvimento de um sistema supervisório Web para o monitoramento de processos industriais, integrando o protocolo OPC-UA a tecnologias modernas de desenvolvimento Web. O sistema implementou com êxito um pipeline completo de aquisição de dados, desde a simulação de sensores até a visualização em dashboards interativos.

Os principais resultados alcançados foram:

- Implementação de um pipeline funcional OPC-UA → Node-RED → Express → Next.js;
- Dashboard de monitoramento em tempo real com gráficos de tendência;
- Sistema de alarmística com geração automática, reconhecimento e histórico;
- Telas de histórico de processo com análise estatística;
- CRUD completo de usuários, dispositivos e sensores;
- Capacidade de escrita bidirecional via OPC-UA.

Como trabalhos futuros, propõe-se:

- Integração com banco de dados de séries temporais para persistência do histórico;
- Implementação de comunicação via WebSockets para redução de latência;
- Inclusão de análise preditiva baseada em algoritmos de aprendizado de máquina;
- Implantação em ambiente de nuvem para acesso remoto;
- Integração com controladores industriais reais (CLPs) via OPC-UA.

Agradecimentos

Agradeço à instituição de ensino SENAI e ao professor orientador Gabriel Claro pela orientação e suporte no desenvolvimento deste trabalho, bem como pelo conteúdo ministrado na disciplina de Interfaces Industriais, que fundamentou este projeto.

Referências

AGUIAR, Claudio. **API REST e Internet das Coisas: conectando o mundo físico ao digital**. [S. l.: s. n.], 2023. DIO.

INTERNATIONAL SOCIETY OF AUTOMATION (ISA). **ISA-101.01-2015: Human Machine Interfaces for Process Automation Systems**. [S. l.], 2015.

NODE-RED. **Node-RED: Low-code programming for event-driven applications.** [S. l.: s. n.], 2024. OpenJS Foundation. Disponível em: <https://nodered.org/>. Acesso em: 20 maio 2025.

OPC FOUNDATION. **OPC Unified Architecture Specification.** [S. l.], 2017.

RODRIGUES, Thaís dos Santos. **Expansão de funcionalidades em sistema IoT de controle de acesso através de API HTTP.** 2024. Monografia (Trabalho de Conclusão de Curso) – Universidade Federal de Pernambuco.